MBAi 460
Modern Software Architectures
Spring 2026

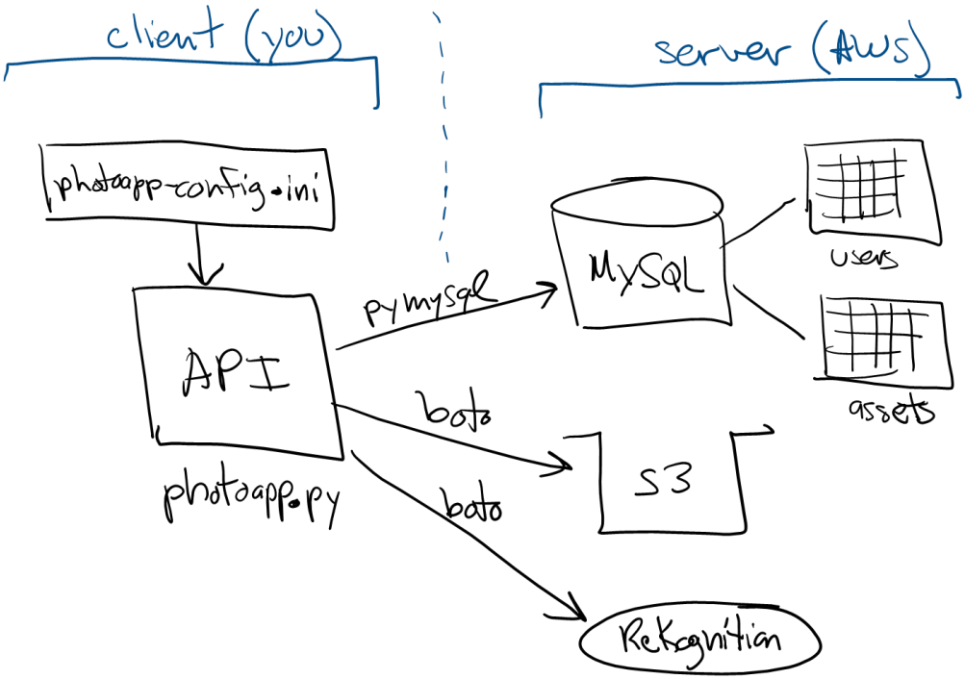
Project #01 – Part 02 (v1.0)

Assignment: PhotoApp with AWS S3, RDS and Rekognition
Submission: via Gradescope
Policy: team-oriented, late work is accepted
Complete By: Monday April 20th @ 11:59pm CST
Late submissions: see syllabus for late policy… No submissions accepted after Thursday 04/23 @ 11:59pm

Pre-requisites: Project #01, Part 01; Relational databases, SQL, Lab 02

Overview

In part 01 you built most of the AWS infrastructure needed for our ***photo-based application***: a MySQL relational database for tracking users and assets, and an S3 bucket for image storage. The goal here in part 02 is to build out a Python-based API for our photo app. This API will use the ***boto*** and ***pymysql*** libraries to upload images, download images, analyze images using the AI-based Rekognition service, and manipulate the database. The end result will be a set of functions in "photoapp.py" that can be used to build a desktop, web, or mobile client for image storage.



Working as part of a team?

This is a non-trivial programming assignment, though completely within the realm of current AI technology which you are encouraged to use. You can work independently or in a team (max size 4), it's completely up to you. If you plan to work as part of a team, please do the following:

1. *Login to canvas*
2. *Click on "People"*
3. *Click on "Groups"*
4. *Scroll down to the groups named "Project 01 1", "Project 01 2", etc.*
5. *Pick a group that has no members, and add yourself to the group. Have your teammates add themselves to the same group.*

That's it! Note that submissions will be collected via Gradescope, not Canvas. When you submit to Gradescope, only one team member needs to submit; every team member receives the same score.

Downloading the provided files

Locate the "mbai460-client-main" folder representing your docker container. Open a Terminal / PowerShell window in this folder, and run the docker container. Then change (cd) into the **projects** folder, then into the **project01** sub-folder, and then into the **client** sub-folder. Use the *ls* command to list the contents of the folder, and you'll see 8 files:

```
hummel> ./docker/run
client:~$ cd projects
client:~/projects$ cd project01
client:~/projects/project01$ cd client
client:~/projects/project01/client$ ls
01degu.jpg   03rollcloud.jpg  client.py            photoapp.py
02earth.jpg  04sailing.jpg    photoapp-config.ini  tests.py
client:~/projects/project01/client$
```

01degu.jpg
02earth.jpg
03rollcloud.jpg
04sailing.jpg
client.py
photoapp-config.ini
photoapp.py
tests.py

We are providing 4 test images for upload/download, a sample config file "photoapp-config.ini", and three Python files. The most important is “photoapp.py”, which will contain your API functions. The file “client.py” can be used to call your API functions for testing purposes. To write actual unit tests for repeatable, automated testing, modify “tests.py”. [*To ensure that test_01 runs first, be sure to name additional tests test_02, test_03, test_04, …*] If you are working outside Docker, these files can be found online in this folder.

Two of the required API functions are already written for you: ***initialize( )*** and ***get_ping( )***. The *initialize( )* function checks the configuration file and configures the local client --- your laptop --- for AWS access; this function needs to be called once (and only once) before the other API functions are called. The *get_ping( )* function "pings" S3 and your MySQL database server to make sure they are accessible based on the provided configuration information.

The provided "client.py" calls these two functions. Go ahead and run the client code, it's going to fail because the given config file does not contain valid information. Don't try to fix the config file, just run and you should see ----------> Initialization works because the config file exists and is properly formatted. However, *get_ping( )* is supposed to return a tuple (M, N) where M is the # of assets in S3, and N is the # of users in the database. Instead, you'll see error messages for both M and N. That is correct behavior at this point.

```
hummel> ./docker/run
cs310-client:~$ cd projects
cs310-client:~/projects$ cd project01
cs310-client:~/projects/project01$ cd part02
cs310-client:~/projects/project01/part02$ cd release
cs310-client:~/projects/project01/part02/release$ python3 client.py   (1)

**starting**

**initialize**   (2)
True

**get_ping**   (3)
ERROR:root:get_ping.get_M():
ERROR:root:An error occurred (InvalidAccessKeyId) when calling the ListObjects operation: The AWS Access Key Id you provided
does not exist in our records.
ERROR:root:get_dbconn():
ERROR:root:(2003, "Can't connect to MySQL server on 'TODO' ([Errno -2] Name or service not known)")
ERROR:root:get_ping.get_N():
ERROR:root:(2003, "Can't connect to MySQL server on 'TODO' ([Errno -2] Name or service not known)")

M: An error occurred (InvalidAccessKeyId) when calling the ListObjects operation: The AWS Access Key Id you provided does not
 exist in our records.
N: (2003, "Can't connect to MySQL server on 'TODO' ([Errno -2] Name or service not known)")

**done**
```

If you completed Part 01, you should have a valid 'photoapp-config.ini' file --- this is the file you submitted to Gradescope. Find this file and copy it into your **client** folder. Open the config file and check the *user_name* and *user_pwd* values in the [rds] section. Make sure they match the following:

```
user_name = photoapp-read-write
user_pwd = def456!!
```

It's okay if you used a different password for the *photoapp-read-write* user, as long as you save that password in the config file so we can access your database when we test your code. Go ahead and run the client again. If your 'photoapp-config.ini' is correct, *get_ping( )* should now output a value of N = 3 since there are 3 users in your database:

```
cs310-client:~/projects/project01/part02/release$ python3 client.py

**starting**

**initialize**
True

**get_ping**
ERROR:root:get_ping.get_M():
ERROR:root:An error occurred (InvalidAccessKeyId) when calling the ListObjects operation: The AWS Access Key Id you provided
does not exist in our records.

M: An error occurred (InvalidAccessKeyId) when calling the ListObjects operation: The AWS Access Key Id you provided does not
 exist in our records.
N: 3

**done**
```

S3 remains inaccessible because we haven't configured programmatic client-side access to S3; we'll do that soon. There's something else I want to focus on first…

The design of “photoapp.py”

It's time to take a look at the provided API code in “photoapp.py”. The design of this code is somewhat subtle, especially if you are new to proper exception handling. So please take the time to review and understand the code, since you will need to mimic this design as you approach the other API functions. At the top level, “photoapp.py” contains 3 helper functions and 2 API functions:

```
##################################################################
#
# get_dbConn
#
# create and return connection object, based on configuration
# information in app config file. You should call close() on
# the object when you are done.
#
def get_dbConn():
  .
  .

##################################################################
#
# get_bucket
#
# create and return bucket object, based on configuration
# information in app config file. You should call close()
# on the object when you are done.
#
def get_bucket():
  .
  .

##################################################################
#
# get_rekognition
#
# create and return rekognition object, based on configuration
# information in app config file. You should call close() on
# the object when you are done.
#
def get_rekognition():
  .
  .

##################################################################
#
# initialize
#
# Initializes local environment need to access AWS, based on
```

```
# given configuration file and user profiles. Call this function
# only once, and call before calling any other API functions.
#
# NOTE: does not check to make sure we can actually reach and
# login to S3 and database server. Call get_ping() to check.
#
def initialize(config_file, s3_profile, mysql_user):
  .
  .

#####################################################################
#
# get_ping
#
# To "ping" a system is to see if it's up and running. This
# function pings the bucket and the database server to make
# sure they are up and running. Returns a tuple (M, N), where
#
#   M = # of items in the photoapp bucket
#   N = # of users in the photoapp.users table
#
# If an error occurs / a service is not accessible, M or N
# will be an error message. Hopefully the error messages will
# convey what is going on (e.g. no internet connection).
#
def get_ping():
  .
  .
```

The 3 "get" functions --- ***get_dbConn( )***, ***get_bucket( )***, and ***get_rekognition( )*** --- return objects to access your database, S3, and Rekognition. Whenever one of your API functions needs to access a resource in the cloud, the idea is to obtain one of these objects to open a connection, use the object to perform the work you need, and then close the connection when you're done (if required). This pattern of open, use, close allows the cloud resources to be shared more easily across clients (especially when each connection is a paid license, which is often the case with database servers).

To see this in action, let's look at the design of the ***get_ping( )*** API function. First, notice that get_ping( ) is defined in terms of two inner functions, get_M( ) and get_N( ). This is done so that if one fails, we can still execute the other one separately in hope that it might succeed. We saw this earlier when the call to S3 failed but the call to the database server successfully returned N = 3:

```
def get_ping():

  def get_M():
    .
    .
  def get_N():
```

```
  .
  .

#
# we compute M and N separately so that we can do separate exception
# handling, and thus get partial results if one succeeds and one fails:
#
try:
  M = get_M()
except Exception as err:
  M = str(err)

try:
  N = get_N()
except Exception as err:
  N = str(err)

return (M, N)
```

The main body of get_ping( ) first calls ***get_M( )*** to access S3, using *try-except* to catch any exceptions that may occur in the call (and setting M to the error message so the error message can be returned). Then the ***get_N( )*** function is called to access the database server, again using *try-except* to catch and return any exceptions that may occur. After the two calls, the tuple (M, N) is returned. Notice that exception handling with ***try-except*** plays a crucial role here in obtaining the desired behavior of returning two values (M, N) even though one or both computations may fail. You are required to use proper exception handling in all your API functions to ensure correct behavior, even in the presence of failures.

Now let's look more closely at the ***get_M( )*** inner function. This function is responsible for accessing S3, and obtaining the # of assets in the bucket. The design is to obtain a ***bucket*** object, call to get the # of objects, close the bucket connection, and return this value. A *try-except-finally* is used to ensure the bucket is properly closed, even in the presence of exceptions:

```
def get_M():
  try:
    #
    # access S3 and obtain the # of items in the bucket:
    #
    bucket = get_bucket()

    assets = bucket.objects.all()

    M = len(list(assets))
    return M

  except Exception as err:
    logging.error("get_ping.get_M():")
    logging.error(str(err))
    raise
```

```
  finally:
    try:
      bucket.close()
    except:
      pass
```

Notice the *bucket.close( )* is wrapped by *try-except* since closing the connection may fail if there was an error when opening the connection. [*Update: turns out bucket.close( ) is no longer necessary / supported; the underlying bucket object automatically opens and closes connections to S3. So you can actually delete the entire finally block if you want.*]

The design of ***get_N( )*** inner function is similar. In this case we open a connection by getting a ***dbConn*** object, and then create a cursor ***dbCursor*** for executing SQL:

```
def get_N():
  try:
    #
    # create connection to MySQL database server and then
    # execute query to retrieve # of users:
    #
    dbConn = get_dbConn()
    dbCursor = dbConn.cursor()

    sql = """
          SELECT count(userid) FROM users;
          """

    dbCursor.execute(sql)
    row = dbCursor.fetchone()

    #
    # we get back a tuple with one result in it:
    #
    N = row[0]
    return N

  except Exception as err:
    logging.error("get_ping.get_N():")
    logging.error(str(err))
    raise

  finally:
    try:
      dbCursor.close()
    except:
      pass
    try:
```

```
        dbConn.close()
      except:
        pass
```

The function then creates and executes the SQL, closes the cursor and connection, and returns. Once again a *finally* block is used to ensure the objects are closed even in the presence of exceptions. [*Optimization: turns out closing the connection is all you need since closing the connection also closes any open cursors. So if you want you can delete the try-except block that closes the cursor.*]

You'll want to use inner functions when it's necessary to separate exception handling, or to implement retry logic. Let's continue this discussion in the next section…

Retry

Recall in Lab 01 we talked about the notion of "retries" in the context of distributed systems. Computers and our networking infrastructure have become very reliable, but nothing is 100%. For this reason it's common to implement retry logic when making a network request to another computer.

In Lab 01 we learned that Amazon's ***boto*** library provides built-in, customizable support for retries. Unfortunately, it turns out that the ***pymysql*** library does NOT provide support for retry logic. This implies we will have to implement retries ourselves whenever we are interacting with the database server. The good news is that we'll take advantage of Python's ***tenacity*** library to add the necessary retry logic to our code (replacing the hand-written approach we used in Lab 01 for web service calls).

If you look at the code for *get_bucket( )* and *get_rekognition( )*, you'll see that ***boto*** is configured to retry at most 3 times:

```
    s3 = boto3.resource(
           's3',
           region_name=regionname,
           config = Config(
             retries = {
               'max_attempts': 3,
               'mode': 'standard'
             }
           )
         )
```

Unfortunately, there is no similar mechanism for configuring ***pymysql***. In fact, *pymysql* does not support retry logic at all; this burden falls to us. We're going to see how to add retry logic to the *s* function.

The good news is that Python offers a ***decorator*** library that provides retry support. A *decorator* library is a set of functions that are applied by annotating the code instead of calling the functions directly. In Python, decorators are applied using the **@** symbol, above the function in this case:

```
@retry(stop=stop_after_attempt(3),
        wait=wait_exponential(multiplier=1, min=2, max=30),
        reraise=True
      )
def get_N():
```

In this example we are annotating the *get_N( )* function so that when it's called, it will execute multiple times, not just once. In particular, the above annotations yield the following behavior:

1. *If an exception occurs, retry execution, stopping after 3 retries*
2. *Wait an exponential amount of time between retries (in this case in powers of 2: 1, 2, 4, 8, …, 30)*
3. *After 3 retries, raise the exception that occurred during the last execution*

This example is based on the ***tenacity*** Python library. The following import is required for the above example to run successfully:

```
from tenacity import retry, stop_after_attempt, wait_exponential
```

You are required to implement retry logic in all calls involving the ***pymysql*** library; we encourage you to use inner functions and the ***tenacity*** library as shown.

Now let's see the *tenacity* library in action. Here's where we left off earlier: the provided code successfully connects to database server but fails to connect to S3. Go ahead and run the client code:

```
cs310-client:~/projects/project01/part02/release$ python3 client.py

**starting**

**initialize**
True

**get_ping**
ERROR:root:get_ping.get_M():
ERROR:root:An error occurred (InvalidAccessKeyId) when calling the ListObjects operation: The AWS Access Key Id you provided
does not exist in our records.

M: An error occurred (InvalidAccessKeyId) when calling the ListObjects operation: The AWS Access Key Id you provided does not
 exist in our records.
N: 3

**done**
```

Let's cause the database access to fail. In "photoapp.py", find the ***get_N( )*** function and intentionally introduce an error. The easiest way is to make a mistake in the SQL query string. For example, change the "SELECT" keyword to something ridiculous like "SELECTALLTHEUSERS":

```
      sql = """
            SELECTALLTHEUSERS count(userid) FROM users;
            """
```

Now run the client code and we'll get an error when it attempts to execute this query:

```
cs310-client:~/projects/project01/part02/release$ python3 client.py

**starting**

**initialize**
True

**get_ping**
ERROR:root:get_ping.get_M():
ERROR:root:An error occurred (InvalidAccessKeyId) when calling the ListObjects operation: The AWS Access Key Id you provided
does not exist in our records.
ERROR:root:get_ping.get_N(): 1
ERROR:root:(1064, "You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for t
he right syntax to use near 'SELECTALLTHEUSERS count(userid) FROM users' at line 1")

M: An error occurred (InvalidAccessKeyId) when calling the ListObjects operation: The AWS Access Key Id you provided does not
 exist in our records.
N: (1064, "You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right
 syntax to use near 'SELECTALLTHEUSERS count(userid) FROM users' at line 1") 2

**done**
```

We can see that the *get_N( )* function reports the error in step (1), and the error is then raised back to the client code which likewise reports the error in step (2). Now let's see the impact of the *tenacity* library. First, add the necessary import at the top of "photoapp.py":

```
from tenacity import retry, stop_after_attempt, wait_exponential
```

Now add the following decoration above the *get_N( )* function:

```
@retry(stop=stop_after_attempt(3),
        wait=wait_exponential(multiplier=1, min=2, max=30),
        reraise=True
      )
def get_N():
```

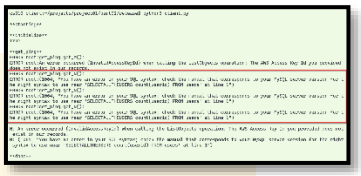
Run the client code again, and now we see *get_N( )* executes 3 times in an attempt to successfully execute the query:

```
cs310-client:~/projects/project01/part02/release$ python3 client.py

**starting**

**initialize**
True

**get_ping**
ERROR:root:get_ping.get_M():
ERROR:root:An error occurred (InvalidAccessKeyId) when calling the ListObjects operation: The AWS Access Key Id you provided
does not exist in our records.
ERROR:root:get_ping.get_N():
ERROR:root:(1064, "You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for t
he right syntax to use near 'SELECTALLTHEUSERS count(userid) FROM users' at line 1")
ERROR:root:get_ping.get_N():
ERROR:root:(1064, "You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for t
he right syntax to use near 'SELECTALLTHEUSERS count(userid) FROM users' at line 1")
ERROR:root:get_ping.get_N():
ERROR:root:(1064, "You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for t
he right syntax to use near 'SELECTALLTHEUSERS count(userid) FROM users' at line 1")

M: An error occurred (InvalidAccessKeyId) when calling the ListObjects operation: The AWS Access Key Id you provided does not
 exist in our records.
N: (1064, "You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right
 syntax to use near 'SELECTALLTHEUSERS count(userid) FROM users' at line 1")

**done**
```

While this is a bad example of why we need retry logic, it demonstrates that *tenacity* + a simple *annotation* can add retry logic to functions, API functions or inner functions. This is the simplest way to fulfill the project requirement of using retry logic in all calls to the *pymysql* library.

Logging

Another aspect of good software engineering is the *logging* of error messages (along with the logging of debug and other forms of program output). You'll notice in the provided API code we are not using print( ) to output error messages, but instead Python's ***logging*** library:



```
    except Exception as err:
      logging.error("get_ping.get_N():")
      logging.error(str(err))
      raise
```

By default, the logging library outputs all messages to the screen --- which is why we've been able to see all the error messages up to this point.

Printing error messages to the screen has two problems: (1) it clutters the screen with hard-to-read output (especially with retry logic), and (2) there's no record of the error when trying to debug the issue later. The advantage of using a logging library is that we can re-configure the library to capture the output in other ways, most commonly in a ***log*** file and thereby solving those 2 problems.

Let's configure the logging library to capture all the output in a text file named "log.txt". Whenever an error occurs, you'll be able to open this file and get more information about the error. Open "client.py" and add the following code near the top of the file:

```
import logging

#
# capture logging output in file 'log.txt'
#
logging.basicConfig(
  filename='log.txt',
  level=logging.INFO,
  format='%(asctime)s - %(levelname)s - %(message)s',
  filemode='w'
)
```

Now, whenever we run the client code, all logging output will be captured in "log.txt". Re-run the client and notice the error messages are no longer output:

```
cs310-client:~/projects/project01/part02/release$ python3 client.py

**starting**

**initialize**
True

**get_ping**

M: An error occurred (InvalidAccessKeyId) when calling the ListObjects operation: The AWS Access Key Id you provided does not
 exist in our records.
N: (1064, "You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right
 syntax to use near 'SELECTALLTHEUSERS count(userid) FROM users' at line 1")

**done**
```

All we see if the client-side output of M and N, which is exactly what we want. However, if we want to see

more detailed information about the errors, we can look in log.txt:

```
cs310-client:~/projects/project01/part02/release$ more log.txt
2026-01-17 06:35:45,826 - INFO - Found credentials in shared credentials file: photoapp-config.ini
2026-01-17 06:35:46,312 - ERROR - get_ping.get_M():
2026-01-17 06:35:46,313 - ERROR - An error occurred (InvalidAccessKeyId) when calling the ListObjects operation: The AWS Acce
ss Key Id you provided does not exist in our records.
2026-01-17 06:35:46,988 - ERROR - get_ping.get_N():
2026-01-17 06:35:46,989 - ERROR - (1064, "You have an error in your SQL syntax; check the manual that corresponds to your MyS
QL server version for the right syntax to use near 'SELECTALLTHEUSERS count(userid) FROM users' at line 1")
2026-01-17 06:35:49,567 - ERROR - get_ping.get_N():
2026-01-17 06:35:49,569 - ERROR - (1064, "You have an error in your SQL syntax; check the manual that corresponds to your MyS
QL server version for the right syntax to use near 'SELECTALLTHEUSERS count(userid) FROM users' at line 1")
2026-01-17 06:35:52,226 - ERROR - get_ping.get_N():
2026-01-17 06:35:52,231 - ERROR - (1064, "You have an error in your SQL syntax; check the manual that corresponds to your MyS
QL server version for the right syntax to use near 'SELECTALLTHEUSERS count(userid) FROM users' at line 1")
```

Notice there's also an "INFO" message in the log file, output by the *boto* library most likely.

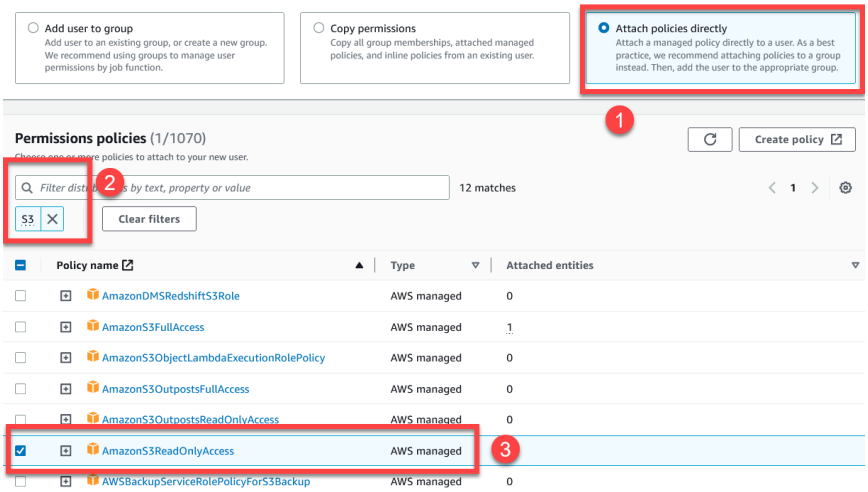
<< this is a good place to take a break before we switch gears back into AWS infrastructure setup >>

Creating S3 users

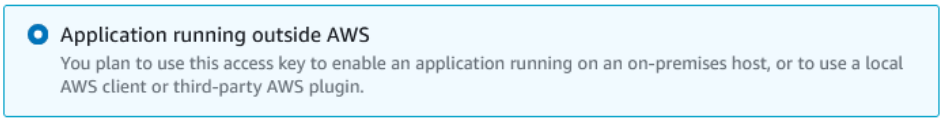
Recall in part 01 we created two fictitious "users" **photoapp-read-only** and **photoapp-read-write** to characterize access to the *photoapp* database. This provides our running program with the necessary identities to access the database. We need to create similar fictitious users to characterize access to *S3*, in this case the "users" **s3readonly** and **s3readwrite** .

Since S3 is directly managed by AWS, we'll use AWS's ***IAM*** service to create the necessary user accounts. The user "**s3readonly**" is easier because IAM has a predefined security policy for this. Here are the steps you followed in part 01 (minus the screenshots) to create the "mycli" user. We need to repeat those steps here:

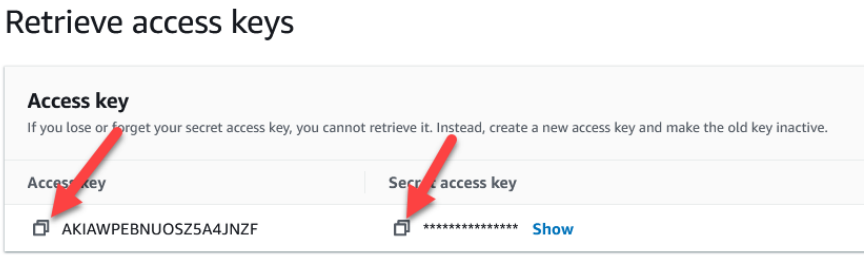
1. Login to AWS management console
2. Search for IAM service
3. Click on Users (left side)
4. Click on "Add users"
5. User name: "**s3readonly**"
6. Next
7. Click on option for "Attach policies directly" and a big list appears…
8. In the filter box type "S3" and press ENTER
9. You should see a much smaller list, check the option for **AmazonS3ReadOnlyAccess**:



10. Next
11. Create User
12. You should be back in the Users display, with “s3readonly” now in the list.
13. Now that the user is created, we need their access and secret keys…
14. Click on the user name, switch to “Security credentials” tab, scroll down to “Access keys”, press “Create access key”, and a list of options will appear. Select “Application running outside AWS” from the list:



15. Next
16. Create access key
17. Now you should see a page like this, where you want to **pause**:



18. Use the little buttons to copy the access and secret keys, and paste them into your “photoapp-config.ini” config file under the section for “s3readonly”, replacing the TODO placeholders.
19. Back in AWS management console, press Done to complete the process.

We need to repeat this process and create another user "**s3readwrite**" that has both read and write permissions to your bucket. Unfortunately AWS doesn't have a predefined policy for this, so first we need to define an appropriate security policy, then we can define the user account. Steps:

1. Back in AWS management console
2. View IAM service if not already
3. Click on Policies (left side)
4. Click on "Create Policy"
5. In the form that appears, switch to the JSON tab and enter the following policy exactly as shown (this JSON is available in this online file) --- change YOUR_BUCKET_NAME to the name of your S3 photoapp bucket that you created in part 01. Be careful when you edit the provided JSON, e.g. note that the JSON has /* after the bucket name in the 2nd case.

Visual editor | JSON

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "s3:ListBucket"
            ],
            "Resource": [
                "arn:aws:s3:::YOUR_BUCKET_NAME"
            ]
        },
        {
            "Effect": "Allow",
            "Action": [
                "s3:PutObject",
                "s3:PutObjectAcl",
                "s3:GetObject",
                "s3:GetObjectAcl",
                "s3:DeleteObject"
            ],
            "Resource": [
                "arn:aws:s3:::YOUR_BUCKET_NAME/*"
            ]
        }
    ]
}
```

6. Click Next: Tags
7. No tags… Click Next: Review
8. You should now be on the Review policy page. Here you want to enter a name that includes "S3" so the policy will be easy to find. How about "S3-YOUR-NAME-Read-Write-Policy":

Review policy

Name* S3-YOUR-NAME-Read-Write-Policy

Use alphanumeric and '+=,.@-_' characters. Maximum 128 characters.

Description

Maximum 1000 characters. Use alphanumeric and '+=,.@-_' characters.

Summary

Filter

Service	Access level	Resource	Request condition
Allow (1 of 370 services) Show remaining 369			
S3	Limited: List, Read, Write, Permissions management	Multiple	None

Tags

Key	Value
No tags associated with the resource.	

9. Click "Create policy" to create the new policy.
10. Back in the Policies list, you should see your new policy at the top of the list.
11. Now create a new user "**s3readwrite**", repeating the steps that start on page 3… The only difference is you'll select your new read-write policy when you get to step #9…
12. Paste the access and secret keys for the new "s3readwrite" user into the "s3readwrite" section of your "photoapp-config.ini" config file, replacing the TODO placeholders.

Let's test things out to make sure all is well. First, the 'photoapp-config.ini' file should be complete --- all the TODO placeholders should be replaced with actual configuration information (make sure ***bucket_name*** has been set as well). In the Terminal / PowerShell window that's running your Docker image, run the client code once more. The ***get_ping( )*** should now run without error:

```
cs310-client:~/projects/project01/part02/release$ python3 client.py

**starting**

**initialize**
True

**get_ping**

M: 20
N: 3

**done**
```

Your value for M will probably be much smaller --- you probably only have 1-2 assets in your bucket. **<u>If M still displays an error message</u>**, check the bucket name in the config file. Then go back to IAM and double-check the JSON you attached to the "s3readwrite" user --- did you change YOUR_BUCKET_NAME to the actual name of your S3 bucket? Is the 2nd bucket name in the JSON followed by "/*" as shown? Check the config file --- did the access keys paste correctly? Is the bucket_name setting correct? If you cannot get it to work, delete "S3readwrite" in IAM and try again. There's only two sources of error: either IAM is not configured properly, or the "photoapp-config.ini" file is incorrect.

Once you have the client code working with "s3readwrite", let's test again with "s3readonly". The client code is currently calling ***initialize( )*** with the S3 user 's3readwrite'. Change this to 's3readonly' and run again; the client code should run without error; if M displays an error, then the config file or the configuration of IAM is wrong for "s3readonly".

When all is working, change the client code back to passing 's3readwrite' in the call to initialize( ). S3 is now properly configured, and you're ready to start building out the API!

WARNING WARNING WARNING!

This is probably a good time to backup your work, in particular your 'photoapp-config.ini' file. If you are backing up your work to Github, you **<u>MUST</u>** use a private repository. The access keys in your config file are essentially passwords to S3, and if your config file is posted to a public repo, two things will happen:

1. *Within minutes, Amazon will send you a nasty-gram email saying one of their access keys was found on a public web site. Since this is a security vulnerability, they will disable the access key and lock your account.*
2. *Hackers may likewise discover your public access key and start using your S3 bucket to store things without your knowledge (and you'll get billed for their usage).*

Amazon has web crawlers looking for public access keys, and will find them within minutes. Hacking groups do the same, and won't be far behind Amazon in detecting them.

<< this is another good place to take a break before we start programming >>

API function #1: get_users

It's time to start implementing the API functions. Add these functions to "photoapp.py", and then test by calling from "client.py" and adding unit tests to "tests.py". A unit test for ***get_users( )*** is already provided. Here's the header comment for your first API function: ***get_users( )***. Feel free to copy-paste this comment into your code:

```
##################################################################
#
# get_users
#
def get_users():
  """
  Returns a list of all the users in the database. Each element
  of the list is a tuple containing userid, username, givenname
  and familyname (in that order). The tuples are ordered by userid,
  ascending. If an error occurs, an exception is raised.

  Parameters
  ----------
  N/A

  Returns
  -------
  a list of all the users, where each element of the list is a tuple
  containing userid, username, givenname, and familyname in that order.
  The list is ordered by userid, ascending. On error an exception is
  raised.
  """
```

You are required to use try-except-finally in your implementation, logging all errors and closing all objects. You are also required to decorate this function so that it performs a max of 3 retries; you can annotate the function directly in this case, an inner function is not required. How to test the retry logic? Introduce an error into the SQL query string, and then check the log afterwards; the error message should appear 3 times.

API function #2: get_images

Here's the header comment for your second API function: ***get_images( )***. Feel free to copy-paste this comment into your code:

```
##################################################################
#
# get_images
#
def get_images(userid = None):
  """
  Returns a list of all the images in the database. Each element
  of the list is a tuple containing assetid, userid, localname
  and bucketkey (in this order). The list is ordered by assetid,
  ascending. If a userid is given, then just the images with that
  userid are returned; validity of the userid is not checked,
  which implies that an empty list is returned if the userid is
  invalid. If an error occurs, an exception is raised.

  Parameters
  ----------
  userid (optional) filters the returned images for just this userid

  Returns
  -------
  a list of images, where each element of the list is a tuple
  containing assetid, userid, localname, and bucketkey in that order.
  The list is ordered by assetid, ascending. If an error occurs,
  an exception is raised.
  """
```

You are required to use try-except-finally in your implementation, logging all errors and closing all objects. You are also required to decorate this function so that it performs a max of 3 retries; you can annotate the function directly in this case, an inner function is not required. How to test the retry logic? Introduce an error into the SQL query string, and then check the log afterwards; the error message should appear 3 times.
[*NOTE: for unit testing, as you define new tests, be sure to name them test_02, test_03, test_04, …*]

API function #3: post_image

The first two API functions are straightforward because they only interact with the database server. The third API function is more interesting because this is the function that uploads images, requiring interaction with both S3 and MySQL. Unlike the previous functions, you cannot simply decorate this function to implement the required retry logic --- this would potentially upload the image 3 times, and recall that boto already supports retry logic (it's only explicitly needed when interacting with MySQL). In this API function

you'll need to use inner functions for the MySQL interaction(s), and decorate those. This is not a suggestion, but a requirement.

Here's the header comment for your third API function: ***post_image( )***. Feel free to copy-paste this comment into your code. [***NOTE****: ignore the mention of the Rekognition AI service, we'll come back and add that functionality later. For now just focus on uploading the image to S3 and updating the assets table.*]

```
##################################################################
#
# post_image
#
def post_image(userid, local_filename):
  """
  Uploads an image to S3 with a unique name, allowing the same local
  file to be uploaded multiple times if desired. A record of this
  image is inserted into the database, and upon success a unique
  assetid is returned to identify this image. The image is also
  analyzed by the Rekognition AI service to label objects within
  the image; the results of this analysis are also saved in the
  database (and can be retrieved later via get_image_labels). If
  an error occurs, an exception is raised. An invalid userid is
  considered a ValueError, "no such userid".

  Parameters
  ----------
  userid for whom we are uploading this image
  local filename of image to upload

  Returns
  -------
  image's assetid upon success, raises an exception on error
  """
```

The image should be stored in S3 in a folder based on the user's *username*; use the provided *userid* to lookup the username (this is also how you'll determine if the *userid* is valid). In S3, folders don't really exist, but are simulated by including a "/" in the bucketkey. A good way to generate a unique bucketkey is to use Python's ***uuid*** module like this:

```
  unique_part = str(uuid.uuid4())
  bucketkey = username + "/" + unique_part + "-" + local_filename
```

Notice the local_filename is also included so the bucketkey is both unique and meaningful (retaining the local filename's extension).

To upload a file to S3, just call the **upload_file( )** function on the ***bucket*** object. Remember the bucket object is already configured to perform at most 3 retries before giving up:

```
bucket.upload_file(local_filename, bucketkey)
```

Think about the order of operations: should you upload first then update the database, or update the database first and then upload? You are not required to solve this problem, i.e. if one operation succeeds and the other fails, you are not required to go back and undo the operation that succeeded. However, if you think about the order of operations, one ordering is "safer" than the other. [*Is it better to update the database but fail to upload the image, or better to upload the image but fail to update the database?*]

Be sure to use a transaction when inserting a new row into the *assets* table. After inserting the new row, you can retrieve the auto-generated assetid by executing this SQL query:

```
sql = """
      SELECT LAST_INSERT_ID() AS assetid;
      """
```

You are required to use try-except-finally in your implementation, logging all errors and closing all objects. You are also required to ensure all code that interacts with MySQL is decorated with retry logic; code interacting with S3 must not be decorated since the required retry logic is already supported by boto.

Test your function by calling from the client code, and then checking the contents of S3 using the AWS Management Console. Test your error handling by passing an invalid userid, an invalid filename, and introducing errors into the SQL query strings; check the log file afterwards.

API function #4: get_image

You just wrote the upload function, now let's write the download function. Here's the header comment for your fourth API function: ***get_image( )***. Feel free to copy-paste this comment into your code:

```
#####################################################################
#
# get_image
#
def get_image(assetid, local_filename = None):
  """
  Downloads the image from S3 denoted by the provided asset. If a
  local_filename is provided, the newly-downloaded file is saved
  with this filename (overwriting any existing file with this name).
  If a local_filename is not provided, the newly-downloaded file
  is saved using the local filename that was saved in the database
  when the file was uploaded. If successful, the filename for the
  newly-downloaded file is returned; if an error occurs then an
  exception is raised. An invalid assetid is considered a
  ValueError, "no such assetid".

  Parameters
  ----------
  assetid of image to download
  local filename (optional) for newly-downloaded image
```

```
  Returns
  -------
  local filename for the newly-downloaded file, or raises an
  exception upon error
  """
```

You'll need to retrieve the bucketkey from the database (confirming the assetid is valid). To download a file from S3, just call the **download_file( )** function on the ***bucket*** object. Remember the bucket object is already configured to perform at most 3 retries before giving up:

```
bucket.download_file(bucketkey, local_filename)
```

You are required to use try-except-finally in your implementation, logging all errors and closing all objects. You are also required to ensure all code that interacts with MySQL is decorated with retry logic; code interacting with S3 must not be decorated since the required retry logic is already supported by boto.

Test your function by calling from the client code, and then checking your local file system to make sure the image was downloaded successfully. Test your error handling by passing an invalid assetid, passing an existing local filename (to make sure the file is overwritten), and introducing errors into the SQL query strings; check the log file afterwards.

API function #5: delete_images

The fifth API function is used to empty the assets table and clear out S3; this is very helpful after a round of testing. Here's the header comment for your fifth API function: ***delete_images( )***. Feel free to copy-paste this comment into your code. [***NOTE****: ignore the mention of "associated labels" for now.*]

```
#################################################################
#
# delete_images
#
def delete_images():
  """
  Delete all images and associated labels from the database and
  S3. Returns True if successful, raises an exception on error.
  The images are not deleted from S3 unless the database is
  successfully cleared; if an error occurs either (a) there are
  no changes or (b) the database is cleared but there may be
  one or more images remaining in S3 (which has no negative
  effect since they have unique names).

  Parameters
  ----------
  N/A

  Returns
```

```
-------
True if successful, raises an exception on error
"""
```

Think about the order of operations; read the header comment again for a pretty big hint. You are required to use try-except-finally in your implementation, logging all errors and closing all objects. You are also required to ensure all code that interacts with MySQL is decorated with retry logic; code interacting with S3 must not be decorated since the required retry logic is already supported by boto.

The quickest way to empty the *assets* table is to truncate it. However, this erases our auto-increment starting at assetid 1001, so that needs to be reset after the truncate. Here's the SQL query string to use, since we also need to disable warnings about foreign keys:

```
sql = """
      SET foreign_key_checks = 0;
      TRUNCATE TABLE assets;
      SET foreign_key_checks = 1;
      ALTER TABLE assets AUTO_INCREMENT = 1001;
      """
```

Be sure to execute this query within the context of a transaction.

To delete all the images from S3, use the ***bucket.delete_objects( )*** function, which can delete up to 1,000 objects at a time; you may assume you'll never have more than 1,000 objects to delete. First build a list of dictionary objects, where each object is of the form { 'Key': bucketkey }. Then you pass this list to the *delete_objects( )* function as follows:

```
bucket.delete_objects(
  Delete={'Objects': objects_to_delete}
)
```

[***NOTE****: the function fails if the list of objects to delete is empty, so check this as a special case before calling.*]

<< this is a good place to take a break before we implement the last two API functions… >>

Rekognition

AWS contains a number of AI-based services. Rekognition is a service that analyzes images to label objects, faces, and more. It's surprisingly easy to use: you pass the service the bucketkey of an image in S3, and it returns a list of object labels, confidence values, etc. Example: if you pass the "degu.jpg" image to *Rekognition*, you'll get back the following list of dictionary objects. Each object will contain a label name --- e.g. 'Animal' --- and a confidence level:

```
[{'Name': 'Animal', 'Confidence': 97.44255828857422, ...},
 {'Name': 'Mammal', 'Confidence': 97.44255828857422, ...},
 {'Name': 'Rodent', 'Confidence': 97.44255828857422, ...},
 {'Name': 'Rat', 'Confidence': 96.63008117675781, ...}]
```

You're going to integrate Rekognition into the API as follows:

1. *Go back and modify **post_image( )** to analyze the image after uploading to S3, storing the analysis results in the database*
2. *Go back and modify **delete_images( )** to eliminate all labels from the database*
3. *Add a **get_image_labels( )** function to the API to retrieve analysis results for a given image*
4. *Add a **get_images_with_label( )** function to the API to search analysis results for a label*

The first step is to extend the ***photoapp*** database to store the results returned by Rekognition. You need to store only two values: the label name (e.g. 'Animal') and the integer confidence value (e.g. 97). Rekognition returns the confidence as a floating-point value, truncate using the *int( )* or *floor( )* function before storing. You'll also need to associate the results with the image that was analyzed; using the ***assetid*** (primary key) is the best approach for relating the analysis results to the image. One table is sufficient, though you may consider using two tables (similar to the design of how movie *genres* are stored in the ***movielens*** database). Write the SQL to create your table(s), and execute using your SQL tool of your choice. [*NOTE: it would be a good idea to save this SQL somewhere so you can recreate your photoapp database if need be.*]

Once the database is ready, go back to your ***post_images( )*** funtion and make the necessary modifications to (1) call the *Rekognition* service and (2) store the analysis results in the database. Here's how to call the Rekognition service:

```
response = rekognition.detect_labels(
              Image={
                'S3Object': {
                  'Bucket': bucket.name,
                  'Name': bucketkey,
                },
              },
              MaxLabels=100,
              MinConfidence=80,
           )
```

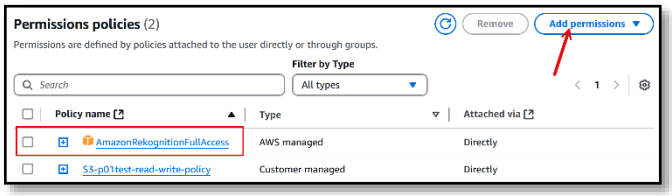
```
labels = response['Labels']
#
# labels is a list of dictionary objects:
#   [{'Name': 'Animal', 'Confidence': 97.44255828857422, 'Instances': [], ...
#
```

We pass the bucketkey of the image to analyze, along with a maximum resultset of 100 labels with a minimum required confidence level of 80%. When you make the call, expect your program to fail however:

```
botocore.errorfactory.AccessDeniedException: An error occurred
(AccessDeniedException) when calling the DetectLabels operation: User:
arn:aws:iam::086636365653:user/s3readwrite is not authorized to perform:
rekognition:DetectLabels because no identity-based policy
allows the rekognition:DetectLabels action
```

The call should fail because your program is running under the user identity "s3readwrite", and that user doesn't have permission to access the Rekognition service. We need to give our program access to Rekognition, much like we gave our program access to S3. You'll recall we did this in part 01 using the IAM service --- go ahead and open the IAM console in AWS. Complete the following steps:

1. *Open IAM*
2. *Click on Users*
3. *Click on user "s3readwrite"*
4. *Under Permissions tab, click "Add permissions"*
5. *Select "Attach policies directly"*
6. *Search for "Rekognition"*
7. *Select "AmazonRekognitionFullAccess", click Next*
8. *Click "Add permissions"*



Now try again and the call to Rekognition should succeed. Once you have the results, execute one or more insert queries to save the results in the database. Use an inner function for interacting with MySQL, and decorate this function with the necessary retry logic. Remember to update the database in the context of a single transaction; a single transaction is more efficient (and appropriate) for bulk updates.

The changes to ***delete_images( )*** are easier: modify the SQL query to truncate the appropriate table(s) so all analysis results are deleted.

API function #6: get_image_labels

Only two API functions left! Now that the Rekognition service is in place, we need a function to return the analysis results for an image. Here's the header comment for your sixth API function: ***get_image_labels( )***. Feel free to copy-paste this comment into your code:

```
#####################################################################
#
# get_image_labels
#
```

```
def get_image_labels(assetid):
  """
  When an image is uploaded to S3, the Rekognition AI service is
  automatically called to label objects in the image. Given the
  image assetid, this function retrieves those labels. In
  particular this function returns a list of tuples. Each tuple
  is of the form (label, confidence), where label is a string
  (e.g. 'sailboat') and confidence is an integer (e.g. 90).
  The tuples are ordered by label, ascending. If an error occurs
  an exception is raised; an invalid assetid is considered a
  ValueError, "no such assetid".

  Parameters
  ----------
  image assetid to retrieve labels for

  Returns
  -------
  a list of labels identified in the image, where each element
  of the list is a tuple of the form (label, confidence) where
  label is a string and confidence is an integer. If an error
  occurs an exception is raised; an invalid assetid is considered
  a ValueError, "no such assetid".
  """
```

You are required to use try-except-finally in your implementation, logging all errors and closing all objects. You are also required to decorate appropriately to add the necessary retry logic. Be sure to test with a valid assetid, an invalid assetid, and by introducing errors into the SQL query strings (checking the log afterwards).

API function #7: get_images_with_label

The last API function is a search function: given a label such as “boat”, this function searches across all analysis results to find images with labels that contain “boat” --- this could be “sailboat” or “powerboat” or just “boat”. The search is thus a substring-based search, and is case-insensitive. This is easily done using the SQL ***like*** operator with a search pattern of ‘%label%’. Here’s the header comment for your seventh and final API function: ***get_images_with_label( )***. Feel free to copy-paste this comment into your code:

```
#####################################################################
#
# get_images_with_label
#
def get_images_with_label(label):
  """
  When an image is uploaded to S3, the Rekognition AI service is
  automatically called to label objects in the image. These labels
  are then stored in the database for retrieval / search. Given a
```

```
label (partial such as 'boat' or complete 'sailboat'), this
function performs a case-insensitive search for all images with
this label. The function returns a list of images, where each
element of the list is a tuple of the form (assetid, label,
confidence). The list is returned in order by assetid, and for
all elements with the same assetid, ordered by label. If an
error occurs, an exception is raised.

Parameters
----------
label to search for, this can be a partial word (e.g. 'boat')

Returns
-------
a list of images that contain this label, even partial matches.
Each element of the list is a tuple (assetid, label, confidence)
where assetid identifies the image, label is a string, and
confidence is an integer. The list is returned in order by
assetid, and for all elements with the same assetid, ordered
by label. If an error occurs, an exception is raised.
"""
```

As with all dynamic SQL queries, use a placeholder in the query string and then pass the necessary parameter to the *execute( )* function. In this case, the search pattern is specified in the call to the execute function, not in the query string itself:

```
    search_pattern = "%" + str(label) + "%"

    dbCursor.execute(sql, [search_pattern])
```

Use two-level sorting (an ORDER BY with two clauses) to sort the results appropriately. You are required to use try-except-finally in your implementation, logging all errors and closing all objects. You are also required to decorate appropriately to add the necessary retry logic.

Grading and Electronic Submission

Your work will be collected and graded using Gradescope. The goal is an autograder score of 70/70; be sure to check the results of your submission, do not simply submit and walk away. If you get a score < 70, look at the Gradescope output, fix your errors, and resubmit; you have unlimited submissions. If you are working in Docker, you can submit your files to gradescope as follows:

```
/gradescope/gs submit 1288073 7983365 *.py *.ini
```

If the above fails, then gradescope is not yet accepting submissions; come back and try again later (Gradescope generally opens a few days before the due date). If you are working outside Docker, you can

submit from your LOCAL computer by viewing your folder locally using Mac/Windows/Linux, open Gradescope (via Canvas or https://gradescope.com), open "Project 01, part02 – API", and then drag-drop your files.

By default, Gradescope records the score of your **LAST SUBMISSION** (not your highest submission score). If you want us to grade a different submission, activate the appropriate submission via your "Submission History". This must be done before the due date.

Pause your RDS server?

Once you have submitted to Gradescope and completed the assignment, this is a good time to select your database instance and under "Actions", select "Stop temporarily" --- this will pause the server, free the assigned VM instance, and reduce the odds of you being charged (AWS has started charging for public IP addresses). However, you'll need to manually restart when you want to resume working in part 02. As noted earlier, you can start and stop your RDS server using the AWS CLI, where the DB_INSTANCE_NAME is something like *mysql-nu-cs310*:

```
aws rds stop-db-instance --db-instance-identifier DB_INSTANCE_NAME
.
aws rds start-db-instance --db-instance-identifier DB_INSTANCE_NAM
```